Java Networking & IPC Practical Questions

Step-by-step VS Code guide with Java source code, commands, expected outputs, explanations, viva questions, and results.

Included questions from this chat: Socket Programming, CRC, Stop-and-Wait ARQ, TCP Daytime Server, Sliding Window, TCP Echo Server, and IPC Message Queue.

1. Implement the basics of socket programming using Java. Create a client-server program to establish communication and exchange messages.

Aim: To implement basic socket programming using Java by creating a client-server program that establishes a connection and exchanges messages.

A server waits for a client, and a client connects to it using a socket. This example uses TCP port 5000.

Step 1: Check Java

In VS Code, open Terminal → New Terminal and run:

```
java -version
javac -version
```

If Java is unavailable, install a JDK and restart VS Code.

Step 2: Create files

```
JavaSocketPractical/
    Server.java
    Client.java
```

Step 3: Server.java

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        int port = 5000;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Server started.");
            System.out.println("Waiting for client...");
            try (Socket socket = serverSocket.accept();
                 BufferedReader input = new BufferedReader(
                     new InputStreamReader(socket.getInputStream()));
                 PrintWriter output = new PrintWriter(
                     socket.getOutputStream(), true)) {
                System.out.println("Client connected!");
                String message;
                while ((message = input.readLine()) != null) {
                    System.out.println("Client: " + message);
                    if (message.equalsIgnoreCase("exit")) {
                        output.println("Goodbye from server!");
                        break;
                    }
                    output.println("Server received: " + message);
                }
                System.out.println("Connection closed.");
            }
        } catch (IOException e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }
}
```

Step 4: Client.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Client {
    public static void main(String[] args) {
        String host = "localhost";
        int port = 5000;
        try (Socket socket = new Socket(host, port);
             BufferedReader input = new BufferedReader(
                 new InputStreamReader(socket.getInputStream()));
             PrintWriter output = new PrintWriter(
                 socket.getOutputStream(), true);
             Scanner scanner = new Scanner(System.in)) {
            System.out.println("Connected to server!");
            while (true) {
                System.out.print("Enter message (exit to stop): ");
                String message = scanner.nextLine();
```

```
                output.println(message);
                String response = input.readLine();
                System.out.println("Server: " + response);
                if (message.equalsIgnoreCase("exit")) break;
            }
            System.out.println("Client disconnected.");
        } catch (IOException e) {
            System.out.println("Client error: " + e.getMessage());
            System.out.println("Make sure the server is running first.");
        }
    }
}
```

Step 5: Compile and run

```
javac Server.java Client.java
# Terminal 1
java Server
# Terminal 2
java Client
```

Run the server first and keep it running. Then open a second terminal and run the client. Type messages; type exit to stop.

Expected output

```
Server: Server started.
Server: Waiting for client...
Client: Connected to server!
Enter message: Hello
Server: Server received: Hello
Enter message: exit
Server: Goodbye from server!
```

Result: Client and server communicate over TCP and exchange messages.

2. Implement the Cyclic Redundancy Check (CRC) using Java and demonstrate error detection.

Aim: To implement CRC in Java and demonstrate how it detects errors during data transmission.

CRC uses modulo-2 binary division with XOR. A zero remainder means no error was detected; a nonzero remainder indicates an error.

Step 1: Create CRC.java

```
public class CRC {
    static String divide(String data, String generator) {
        char[] bits = data.toCharArray();
        int n = generator.length();
        for (int i = 0; i <= bits.length - n; i++) {
            if (bits[i] == '1') {
                for (int j = 0; j < n; j++) {
                    bits[i + j] = (bits[i + j] == generator.charAt(j))
                            ? '0' : '1';
                }
            }
        }
        return new String(bits, bits.length - n + 1, n - 1);
    }

    static boolean hasError(String received, String generator) {
        return !divide(received, generator).matches("0+");
    }

    public static void main(String[] args) {
        String data = "1101011011";
        String generator = "10011";
        System.out.println("Original Data: " + data);
        System.out.println("Generator:     " + generator);
        String appendedData = data + "0".repeat(generator.length() - 1);
        String crc = divide(appendedData, generator);
        String transmitted = data + crc;
        System.out.println("\\nCRC Remainder: " + crc);
        System.out.println("Transmitted Data: " + transmitted);

        System.out.println("\\n--- Test 1: Without Error ---");
        System.out.println("Received Data: " + transmitted);
        System.out.println(hasError(transmitted, generator)
                ? "Error Detected!" : "No Error Detected.");

        System.out.println("\\n--- Test 2: With Error ---");
        char[] corrupted = transmitted.toCharArray();
        corrupted[0] = corrupted[0] == '0' ? '1' : '0';
        String receivedWithError = new String(corrupted);
        System.out.println("Received Data: " + receivedWithError);
        System.out.println(hasError(receivedWithError, generator)
                ? "Error Detected!" : "No Error Detected.");
    }
}
```

Step 2: Compile and run

```
javac CRC.java
java CRC
```

Expected output

```
Original Data: 1101011011
Generator:     10011

CRC Remainder: 1110
Transmitted Data: 11010110111110

--- Test 1: Without Error ---
Received Data: 11010110111110
No Error Detected.

--- Test 2: With Error ---
Received Data: 01010110111110
Error Detected!
```

Viva: CRC means Cyclic Redundancy Check; XOR is used in modulo-2 division; CRC detects but does not itself correct errors.

Result: CRC bits are calculated and a flipped bit is detected.

3. Implement the Stop-and-Wait ARQ protocol using Java and demonstrate frame transmission and acknowledgement.

Aim: To simulate Stop-and-Wait ARQ, including frame transmission, acknowledgements, lost ACK, timeout, and retransmission.

The sender sends one frame and waits for its ACK before sending the next frame.

Step 1: Create StopWaitARQ.java

```
public class StopWaitARQ {
    public static void main(String[] args) {
        int totalFrames = 4;
        int expectedSeq = 0;
        boolean ackLostOnce = false;

        System.out.println("Stop-and-Wait ARQ Protocol");
        System.out.println("--------------------------");

        for (int frame = 1; frame <= totalFrames; frame++) {
            int seq = (frame - 1) % 2;
            boolean acknowledged = false;

            while (!acknowledged) {
                System.out.println("\\nSender: Sending Frame "
                        + frame + " (Seq=" + seq + ")");

                if (seq == expectedSeq) {
                    System.out.println("Receiver: Frame "
                            + frame + " received successfully.");
                    expectedSeq = 1 - expectedSeq;
                } else {
                    System.out.println("Receiver: Duplicate frame detected.");
                }

                if (frame == 2 && !ackLostOnce) {
                    System.out.println("ACK " + seq + " lost!");
                    System.out.println("Sender: Timeout. Retransmitting frame "
                            + frame + ".");
                    ackLostOnce = true;
                    continue;
                }

                System.out.println("Receiver: Sending ACK " + seq);
                System.out.println("Sender: ACK " + seq + " received.");
                acknowledged = true;
            }
        }
        System.out.println("\\nAll frames transmitted successfully.");
    }
}
```

Step 2: Compile and run

```
javac StopWaitARQ.java
java StopWaitARQ
```

Expected behavior

Frame 1 is sent and acknowledged. The first ACK for Frame 2 is deliberately lost, so the sender retransmits Frame 2. The receiver recognizes the duplicate and acknowledges it again. Frames 3 and 4 then complete.

Viva: ACK confirms reception; lost ACK causes timeout and retransmission; sequence numbers help identify duplicates.

Result: Stop-and-Wait ARQ behavior is demonstrated by simulation.

4. Develop a TCP Daytime Server program using C/Java that sends the current date and time to a client.

Aim: To create a TCP server that sends the current date and time to a client.

The Java server listens on port 5000 and sends a formatted date-time string to each connecting client.

Step 1: Create files

```
DaytimeServer.java
DaytimeClient.java
```

Step 2: DaytimeServer.java

```
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class DaytimeServer {
    public static void main(String[] args) {
        int port = 5000;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Daytime Server started.");
            System.out.println("Waiting for client...");
            while (true) {
                try (Socket socket = serverSocket.accept();
                     PrintWriter output = new PrintWriter(
                             socket.getOutputStream(), true)) {
                    System.out.println("Client connected.");
                    String dateTime = LocalDateTime.now().format(
                            DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss"));
                    output.println("Current Date and Time: " + dateTime);
                    System.out.println("Date and time sent: " + dateTime);
                }
            }
        } catch (Exception e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }
}
```

Step 3: DaytimeClient.java

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.Socket;

public class DaytimeClient {
    public static void main(String[] args) {
        String host = "localhost";
        int port = 5000;
        try (Socket socket = new Socket(host, port);
             BufferedReader input = new BufferedReader(
                     new InputStreamReader(socket.getInputStream()))) {
            System.out.println("Connected to Daytime Server.");
            System.out.println(input.readLine());
        } catch (Exception e) {
            System.out.println("Client error: " + e.getMessage());
            System.out.println("Start the server before the client.");
        }
    }
}
```

Step 4: Compile and run

```
javac DaytimeServer.java DaytimeClient.java
# Terminal 1
java DaytimeServer
# Terminal 2
java DaytimeClient
```

Expected output

```
Connected to Daytime Server.
Current Date and Time: 28-10-2026 09:00:00

(The date/time will match your computer clock.)
```

Troubleshooting: If 'Address already in use' appears, stop the old server with Ctrl+C or change the port in both files.

Result: The server sends the current date and time over TCP.

5. Implement the Sliding Window protocol using Java and demonstrate transmission of multiple frames and acknowledgements.

Aim: To simulate the Sliding Window protocol by transmitting multiple frames and displaying acknowledgements.

The sender can send several frames within a window before moving the window forward after ACKs.

Step 1: Create SlidingWindow.java

```
public class SlidingWindow {
    public static void main(String[] args) {
        int totalFrames = 8;
        int windowSize = 3;
        int base = 1;

        System.out.println("Sliding Window Protocol");
        System.out.println("----------------------");
        System.out.println("Total Frames: " + totalFrames);
        System.out.println("Window Size: " + windowSize);

        while (base <= totalFrames) {
            int end = Math.min(base + windowSize - 1, totalFrames);
            System.out.println("\\nSender: Sending frames " + base + " to " + end);

            for (int frame = base; frame <= end; frame++)
                System.out.println("Sender -> Receiver: Frame " + frame);

            for (int frame = base; frame <= end; frame++)
                System.out.println("Receiver -> Sender: ACK " + frame);

            base = end + 1;
            System.out.println("Window slides forward.");
        }
        System.out.println("\\nAll frames transmitted successfully.");
    }
}
```

Step 2: Compile and run

```
javac SlidingWindow.java
java SlidingWindow
```

Expected output

```
Sliding Window Protocol
----------------------
Total Frames: 8
Window Size: 3

Sender: Sending frames 1 to 3
Sender -> Receiver: Frame 1
Sender -> Receiver: Frame 2
Sender -> Receiver: Frame 3
Receiver -> Sender: ACK 1
Receiver -> Sender: ACK 2
Receiver -> Sender: ACK 3
Window slides forward.

Sender: Sending frames 4 to 6
Sender -> Receiver: Frame 4
Sender -> Receiver: Frame 5
Sender -> Receiver: Frame 6
Receiver -> Sender: ACK 4
Receiver -> Sender: ACK 5
Receiver -> Sender: ACK 6
Window slides forward.

Sender: Sending frames 7 to 8
Sender -> Receiver: Frame 7
Sender -> Receiver: Frame 8
Receiver -> Sender: ACK 7
Receiver -> Sender: ACK 8
Window slides forward.

All frames transmitted successfully.
```

Viva: Window size is the maximum number of outstanding frames; ACK confirms reception; sliding windows improve efficiency compared with Stop-and-Wait.

Result: Multiple frame transmission, acknowledgements, and window movement are simulated.

6. Develop a TCP Echo Server program using C/Java that receives a message from a client and returns it.

Aim: To create a TCP Echo Server that sends each received client message back unchanged.

The server listens on port 5001 and echoes messages. The word exit ends the connection.

Step 1: Create files

```
EchoServer.java
EchoClient.java
```

Step 2: EchoServer.java

```
import java.io.*;
import java.net.*;

public class EchoServer {
    public static void main(String[] args) {
        int port = 5001;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("TCP Echo Server started.");
            System.out.println("Waiting for client...");
            try (Socket socket = serverSocket.accept();
                 BufferedReader input = new BufferedReader(
                     new InputStreamReader(socket.getInputStream()));
                 PrintWriter output = new PrintWriter(
                     socket.getOutputStream(), true)) {
                System.out.println("Client connected.");
                String message;
                while ((message = input.readLine()) != null) {
                    System.out.println("Client: " + message);
                    if (message.equalsIgnoreCase("exit")) {
                        output.println("Goodbye!");
                        break;
                    }
                    output.println(message);
                }
                System.out.println("Client disconnected.");
            }
        } catch (IOException e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }
}
```

Step 3: EchoClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class EchoClient {
    public static void main(String[] args) {
        String host = "localhost";
        int port = 5001;
        try (Socket socket = new Socket(host, port);
             BufferedReader input = new BufferedReader(
                 new InputStreamReader(socket.getInputStream()));
             PrintWriter output = new PrintWriter(
                 socket.getOutputStream(), true);
             Scanner scanner = new Scanner(System.in)) {
            System.out.println("Connected to Echo Server.");
            while (true) {
                System.out.print("Enter message (exit to stop): ");
                String message = scanner.nextLine();
                output.println(message);
                System.out.println("Echo from server: " + input.readLine());
                if (message.equalsIgnoreCase("exit")) break;
            }
        } catch (IOException e) {
            System.out.println("Client error: " + e.getMessage());
            System.out.println("Start the server first.");
        }
    }
}
```

Step 4: Compile and run

```
javac EchoServer.java EchoClient.java
# Terminal 1
java EchoServer
# Terminal 2
java EchoClient
```

Expected output

```
Connected to Echo Server.
Enter message (exit to stop): Hello Server
Echo from server: Hello Server
```

```
Enter message (exit to stop): How are you?
Echo from server: How are you?
Enter message (exit to stop): exit
Echo from server: Goodbye!
```

Result: The server receives messages and returns the same text to the client.

7. Implement an IPC Message Queue program using Java to exchange messages between processes.

Aim: To simulate an Inter-Process Communication message queue using Java and exchange messages between two separate processes.

This practical uses a shared file with file locks as a simple file-backed queue. It is not a native OS message queue.

Step 1: Create project

```
IPCMessageQueue/
    MessageQueue.java
    messages.txt (created automatically)
```

Step 2: MessageQueue.java

```
import java.io.*;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.channels.FileLock;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.Scanner;

public class MessageQueue {
    private static final Path QUEUE = Paths.get("messages.txt");

    static void sendMessage(String message) throws IOException {
        byte[] data = (message + "\n").getBytes(StandardCharsets.UTF_8);
        try (FileChannel channel = FileChannel.open(QUEUE,
                StandardOpenOption.CREATE, StandardOpenOption.READ,
                StandardOpenOption.WRITE);
             FileLock lock = channel.lock()) {
            channel.position(channel.size());
            channel.write(ByteBuffer.wrap(data));
        }
    }

    static String receiveMessages() throws IOException {
        try (FileChannel channel = FileChannel.open(QUEUE,
                StandardOpenOption.CREATE, StandardOpenOption.READ,
                StandardOpenOption.WRITE);
             FileLock lock = channel.lock()) {
            long size = channel.size();
            if (size == 0) return "";
            ByteBuffer buffer = ByteBuffer.allocate((int) size);
            channel.position(0);
            while (buffer.hasRemaining() && channel.read(buffer) != -1) { }
            String messages = new String(buffer.array(), 0, buffer.position(),
                    StandardCharsets.UTF_8);
            channel.truncate(0);
            return messages;
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 1 ||
                !(args[0].equalsIgnoreCase("send") ||
                  args[0].equalsIgnoreCase("receive"))) {
            System.out.println("Usage: java MessageQueue send");
            System.out.println("   or: java MessageQueue receive");
            return;
        }

        if (args[0].equalsIgnoreCase("send")) {
            Scanner scanner = new Scanner(System.in);
            System.out.println("Sender process started.");
            System.out.println("Type a message; type 'exit' to stop.");
            while (true) {
                System.out.print("Enter message: ");
                String message = scanner.nextLine();
                if (message.equalsIgnoreCase("exit")) break;
                sendMessage(message);
                System.out.println("Message added to queue.");
            }
            System.out.println("Sender process stopped.");
        } else {
            System.out.println("Receiver process started.");
            System.out.println("Waiting for messages...");
            while (true) {
                String messages = receiveMessages();
                if (!messages.isEmpty()) {
                    for (String message : messages.split("\\\\R")) {
                        if (!message.isEmpty())
```

```
                            System.out.println("Received: " + message);
                        }
                    }
                    Thread.sleep(500);
                }
            }
        }
    }
```

Step 3: Compile

```
javac MessageQueue.java
```

Step 4: Start receiver in Terminal 1

```
java MessageQueue receive
```

Step 5: Start sender in Terminal 2

```
java MessageQueue send
```

Step 6: Example output

Receiver terminal:

```
Receiver process started.
Waiting for messages...
Received: Hello Receiver
Received: How are you?
```

Sender terminal:

```
Sender process started.
Type a message; type 'exit' to stop.
Enter message: Hello Receiver
Message added to queue.
Enter message: How are you?
Message added to queue.
Enter message: exit
Sender process stopped.
```

The receiver checks the queue every 500 ms. Stop it with Ctrl+C. File locking helps prevent simultaneous access to the shared file.

Step 7: Viva questions

What is IPC?
Inter-Process Communication allows processes to exchange data.

What is a message queue?
A queue stores messages until another process retrieves them.

Why use file locking?
It helps prevent the sender and receiver from modifying the shared queue file simultaneously.

Is this a native OS message queue?
No. It is a file-backed simulation. Native queues require OS-specific APIs or a library.

Result: Two separate Java processes exchange messages through a shared file-backed queue.

General VS Code troubleshooting

'javac' is not recognized: Install a JDK and configure PATH, then restart VS Code.

Connection refused: Start the server before the client.

Address already in use: Stop the previous server with Ctrl+C or change the port in both client and server files.

Could not find or load main class: Run commands from the project folder and compile the Java files first.